TEXORA · LECTURE SHEET

Programming Concepts

How a problem becomes a program: the steps of problem solving, what makes an algorithm good, pseudocode and flowcharts as planning tools, the three control structures, and the life cycle that carries a program from idea to maintained software.

CSE-2105 · FUNDAMENTALS OF COMPUTING · TOPIC 5 · LEVEL 2 TERM 1

1. From problem to program

A computer solves nothing by itself. It follows, exactly and at great speed, instructions that a person has designed. **Programming** is the work of designing those instructions so that a given problem is solved correctly for every valid input, not only for the example in front of us. Beginners often rush to typing code; experienced programmers know that most of the real work happens before any code is written, in understanding the problem and planning the solution.

A disciplined approach follows these problem solving steps:

1. **Define the problem.** State clearly what is given (the inputs), what must be produced (the outputs) and what rules or limits apply. Half of all programming failures are really failures to understand what was asked.
2. **Analyse and plan.** Work the problem by hand on small examples. Choose a method — a formula, a search, a step by step procedure — and check it against the examples.
3. **Design the algorithm.** Write the method as an exact, ordered procedure, using pseudocode or a flowchart (Sections 3 and 4).
4. **Code the program.** Translate the design into a programming language, following that language’s rules exactly.
5. **Test and debug.** Run the program on test inputs whose correct answers are already known, including awkward edge cases such as zero, negative values or the largest allowed value. Find and remove errors (Section 8).
6. **Document and maintain.** Record what the program does and how, so that it can be understood, corrected and improved later — often by a different person.

2. Algorithms: definition and properties

An **algorithm** is a finite sequence of clear, unambiguous steps that solves a problem: given valid input, it carries out the steps and produces the required output, then stops. The word comes from the name of the ninth century mathematician al-Khwarizmi, whose systematic procedures for arithmetic are the ancestors of the idea. A cooking recipe is an everyday analogy, but a true algorithm is stricter than a recipe: nothing may be left to guesswork.

A good algorithm has these properties:

PROPERTY	MEANING
Input	It takes zero or more clearly defined inputs from outside.
Output	It produces at least one output — the answer the problem asked for.
Definiteness	Every step is precise and unambiguous; two careful readers would carry it out in exactly the same way.
Finiteness	It must finish after a finite number of steps for every valid input. A procedure that can run forever is not an algorithm.
Effectiveness	Every step is basic enough to actually be carried out, in finite time, by a person with paper and pencil or by a machine.
Generality	It solves a whole class of problems (any two numbers, any list length), not only one fixed example.

Two further qualities separate a merely correct algorithm from a good one. **Correctness** means it gives the right answer for every valid input, including edge cases. **Efficiency** means it uses a sensible amount of time and memory; an algorithm that finds a name in a list of a million entries one by one and an algorithm that halves the list at each step are both correct, but they are not equally useful.

3. Pseudocode

Pseudocode is a halfway language between plain English and a real programming language: the logic of the program written in short, structured statements that a human can read easily, without the strict punctuation rules of any actual language. It lets the designer concentrate on *what the steps are* before worrying about *how one language spells them*. Conventions differ slightly between books, but a widely used set is:

- Write one step per line, in the order the steps will run, and number or indent them to show structure.
- Use capitalised keywords for the main actions: `INPUT`, `OUTPUT` (or `PRINT`), `SET`, `IF ... THEN ... ELSE ... ENDIF`, `WHILE ... ENDWHILE`, `FOR ... ENDFOR`.

- Use an arrow or equals sign for assignment: `total ← total + mark` means “the new value of total is the old value plus mark”.
- Indent the steps that belong inside a decision or a loop, so the structure can be seen at a glance.
- Begin with `BEGIN` (or `START`) and finish with `END` (or `STOP`).

Pseudocode for adding two numbers and printing the result:

```
BEGIN
  INPUT a, b
  SET sum ← a + b
  OUTPUT sum
END
```

There is no compiler for pseudocode, so it cannot be run; its job is communication and planning. If two students read the same pseudocode and understand different procedures, the pseudocode — not the students — is at fault.

4. Flowcharts

A **flowchart** shows the same algorithm as a diagram: boxes of standard shapes joined by arrows called flow lines. Because the shapes are standardised, a flowchart drawn in Dhaka can be read in any other country. The six symbols needed for this course are:

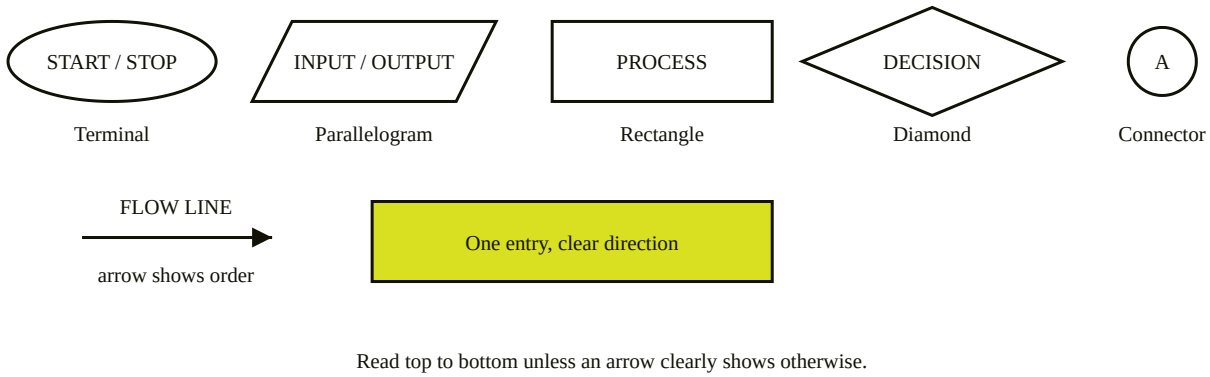


Figure 1 — The standard flowchart symbols

The **terminal** symbol (a rounded oval) marks the start and the end. The **parallelogram** marks input and output. The **rectangle** marks a process or calculation. The **diamond** marks a decision: a question with two exits, usually labelled Yes and No (or True and False). **Flow lines** are arrows showing the order of execution. A small **connector** circle, labelled with a letter, joins a flow line that continues elsewhere on the page, avoiding long crossing arrows. Decisions are the only symbol with two exits; every other symbol has one entry and one exit, which keeps charts readable.

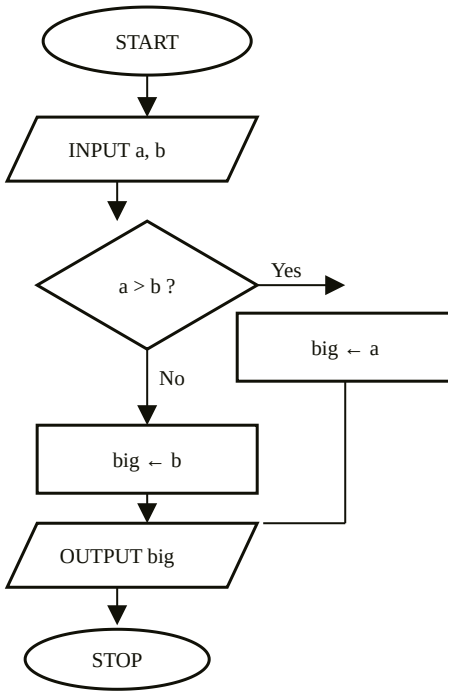


Figure 2 — Flowchart to find the larger of two numbers

Reading Figure 2: the two numbers enter, the diamond asks whether `a > b`. On the Yes path `big` receives `a`; on the No path `big` receives `b`. Both paths rejoin before the output, so the chart prints exactly one value — the larger — and stops. Tracing it with a = 9, b = 4 gives 9; tracing with a = 3, b = 7 gives 7; tracing with a = 5, b = 5 gives 5 by the No path, which is still correct because the two values are equal.

5. The three control structures

A classic result of programming theory says that every algorithm can be built from only three patterns of control. Master these three and every program in this course is a combination of them.

Sequence. Steps run one after another, in written order, each exactly once. Adding two numbers and printing the sum (Section 3) is pure sequence. **Selection.** A condition chooses which steps run. The `IF ... THEN ... ELSE` form runs one block when the condition is true and another when it is false:

```
IF mark >= 40 THEN
  OUTPUT "Pass"
ELSE
  OUTPUT "Fail"
ENDIF
```

Iteration (looping). A block of steps repeats while a condition holds (`WHILE`), or a counted number of times (`FOR`). Printing the numbers 1 to 5:

```
FOR count ← 1 TO 5
  OUTPUT count
ENDFOR
```

The same job with a `WHILE` loop needs the counter managed by hand — set it before the loop, change it inside the loop:

```
SET count ← 1
WHILE count <= 5
  OUTPUT count
  SET count ← count + 1
ENDWHILE
```

Forgetting the last line inside the `WHILE` version is the classic beginner error: the condition never changes, the loop never ends, and the algorithm breaks its own finiteness rule. A loop must always contain some step that moves it towards finishing.

6. Worked algorithm examples

WORKED EXAMPLE (A)

Sum of the first n natural numbers.

```
BEGIN
  INPUT n
  SET sum ← 0
  FOR i ← 1 TO n
    SET sum ← sum + i
  ENDFOR
  OUTPUT sum
END
```

Trace for n = 5: sum starts at 0; after i = 1, sum = 1; after i = 2, sum = 3; after i = 3, sum = 6; after i = 4, sum = 10; after i = 5, sum = 15. Output: **15**.
Check by formula: the sum of the first n natural numbers is S = n(n + 1)/2. For n = 5: S = 5 × 6/2 = **15** ✓. The formula and the loop agree — two independent methods giving one answer is the strongest simple check a student can make.

WORKED EXAMPLE (B)

Factorial of a number. The factorial n! is the product 1 × 2 × ... × n, with 0! defined as 1.

```
BEGIN
  INPUT n
  SET fact ← 1
  FOR i ← 1 TO n
    SET fact ← fact * i
  ENDFOR
  OUTPUT fact
END
```

Trace for n = 5: fact runs 1, 2, 6, 24, 120 — output **120**. Note the starting value: the accumulator for a product must start at 1, not 0, because multiplying by 0 would destroy every later step. Choosing the wrong starting value is a logical error, discussed in Section 8.

WORKED EXAMPLE (C)

Checking whether a number is prime. A prime number greater than 1 has no divisor other than 1 and itself, so test every possible divisor from 2 up to n − 1; if any divides n exactly, the number is not prime.

```
BEGIN
  INPUT n
  SET prime ← true
  IF n < 2 THEN SET prime ← false ENDIF
  FOR d ← 2 TO n - 1
    IF n MOD d = 0 THEN
      SET prime ← false
    ENDIF
  ENDFOR
  IF prime = true THEN OUTPUT "Prime" ELSE OUTPUT "Not prime" ENDIF
END
```

Trace for n = 7: d takes 2, 3, 4, 5, 6; none divides 7 exactly, so the output is **Prime**. Trace for n = 9: at d = 3, 9 MOD 3 = 0, so prime becomes false and the output is **Not prime**. (A faster version tests divisors only up to the square root of n, because a divisor larger than the square root must pair with one smaller than it — an efficiency improvement that leaves the answer unchanged.)

7. The program development life cycle

Real programs — especially those other people will depend on — are built through a recognised cycle of stages, the **program development life cycle**:

STAGE	WHAT HAPPENS

1. Problem definition and analysis	The requirement is agreed and studied: inputs, outputs, rules, users and limits. Feasibility is judged before money is spent.
2. Design	The solution is planned with algorithms, pseudocode, flowcharts and sketches of screens and data. Design errors found here are cheap to fix.
3. Coding	Programmers write the source code in the chosen language, following the design and the language rules.
4. Testing and debugging	The program is run against planned test data: normal cases, boundary cases and invalid inputs. Errors are located and corrected, and the tests are repeated.
5. Implementation	The finished system is installed for its real users, data is converted, and users are trained.
6. Documentation	User guides and technical notes are completed, so the system can be operated and understood without its original author present.
7. Maintenance	After release the program is corrected, adapted to new rules and improved. For long lived software this is the longest and most expensive stage of all.

Notice how this cycle generalises the six problem solving steps of Section 1: the same discipline, scaled up from a classroom exercise to a system that a factory or a bank will run for years.

8. Levels of language and types of error

As background from earlier topics: **low level** languages (machine code and assembly language) speak almost directly to the hardware — fast and fully in control, but hard to write, tied to one processor family and unforgiving of mistakes. **High level** languages (such as C, the subject of the next sheet) use English like words and mathematical notation, are translated by a compiler or interpreter, and let the programmer think about the problem rather than the processor. Almost all modern application programming is done in high level languages.

Finally, learn to name errors correctly, because the name tells you where to look:

ERROR TYPE	MEANING	EXAMPLE
Syntax error	A rule of the language is broken. The translator refuses to produce a working program.	A missing semicolon or a misspelled keyword in C.
Runtime error	The program starts but fails while running, on some particular input or situation.	Dividing by zero, or asking for more memory than exists.

Logical error	The program runs and finishes, but gives the wrong answer. The hardest kind to find, because nothing announces it.	Starting a product accumulator at 0, or using > where >= was needed.

The only reliable defence against logical errors is testing with inputs whose answers are already known by an independent method — exactly the formula check used for the sum algorithm in Section 6.

Glossary

TERM	MEANING
Algorithm	A finite sequence of clear, unambiguous steps that solves a problem and then stops.
Pseudocode	Program logic written in short structured statements between plain English and real code, used for planning.
Flowchart	A diagram of an algorithm using standard symbols joined by flow lines.
Selection	The control structure that chooses which steps run, based on a condition.
Iteration	The control structure that repeats a block of steps; also called looping.
Trace	A step by step record of variable values as an algorithm runs on chosen test data.
Debugging	The work of finding and removing errors from a program.
Syntax error	A broken language rule, caught by the translator before the program can run.
Logical error	An error in reasoning: the program runs to the end but produces a wrong answer.

Key points & summary

- Understand the problem and plan the solution before writing any code; most failures begin as misunderstood requirements.
- An algorithm must have input, output, definite steps, finiteness, effectiveness and generality.
- Pseudocode plans logic in readable statements; a flowchart plans it in standard symbols, with the diamond as the only two exit symbol.
- Every algorithm is built from sequence, selection and iteration; every loop must contain a step that moves it towards finishing.

- Trace tables on small data, checked against an independent formula where one exists, are the basic correctness test.
- The development life cycle runs analysis, design, coding, testing, implementation, documentation and maintenance.
- Syntax errors stop translation, runtime errors stop execution, and logical errors silently give wrong answers.

Practice

Answer in your own words; brief model answers follow.

1. List the properties a procedure must have to count as an algorithm, and state which property is broken by a loop whose condition never becomes false.
2. Write pseudocode that inputs three marks, calculates their average, and prints the average.
3. Draw, in words or symbols, the flowchart shapes you would use for: starting a program, reading a value, adding two values, and asking whether the total exceeds 100.
4. Trace the factorial algorithm of this sheet for n = 4, giving the value of fact after every pass of the loop.
5. A program to calculate wages uses the hourly rate where the number of hours was intended, yet runs without any complaint. What type of error is this, and how would testing reveal it?
6. Place these life cycle stages in order and give the purpose of each in one line: coding, maintenance, design, testing, analysis.

Answer key

1. Input, output, definiteness, finiteness, effectiveness and generality. A loop that never ends breaks finiteness.
2. BEGIN; INPUT m1, m2, m3; SET average ← (m1 + m2 + m3) / 3; OUTPUT average; END. (Any equivalent clear form is acceptable.)
3. Start: terminal oval. Reading a value: parallelogram (input/output). Adding: rectangle (process). Asking about 100: diamond (decision), with Yes and No exits.
4. fact starts at 1. After i = 1: 1. After i = 2: 2. After i = 3: 6. After i = 4: 24. Output 24.
5. A logical error: the program is valid and finishes, but the reasoning is wrong. Planned test data with a hand calculated wage would show the program’s answer differing from the known answer.
6. Analysis (agree and study the requirement), design (plan the solution in algorithms and diagrams), coding (write the source code), testing (run planned test data and remove errors), maintenance (correct and improve the program after release).